CMP417 CW2 – Machine Learning

CREATING A NETWORK PACKET CLASSIFIER

Zack Anderson (1602117@uad.ac.uk) | BSc (Hons) Ethical Hacking | 29/04/2020

Introduction

BACKGROUND

Neural Networks are exceptionally effective for high-dimensionality problems such as network classification, where a packet will have tens or hundreds of features which will be used to classify if they are malicious or not. As the nature of this problem entails, the most appropriate set of algorithms for this task are classification algorithms. These algorithms group items based on common features and qualities using hidden layers of nodes, each of which will check for a specific feature and determine the appropriate next node to send the input data to (Srivastava, 2014). At the end of this process, the model arrives at a decision, or *classification*, for the input data.

A random forest classifier uses a *"forest"* of decision tree classifiers on sub-samples of the input data and uses averaging to improve predictive accuracy compared to a single decision tree (SciKit-Learn, n.d). , Decision tree's operate similarly to layers of if statements to arrive at a classification, with each node representing a new criterion with 2 outcomes.

NETWORK DATA SET

The data provided for training purposes in this project is an extract from the UNSW NB15 data set (Moustafa & Slay, 2015). This data set provides 82,332 network packets as training data and a further 175,341 network packets as testing data. Each packet has 42 features including the network protocol used (such as TCP, UDP, etcetera), the state of the packet (which is dependent on the protocol used), and the service of the packet (such as HTTP or SSH). Each packet is also labelled with one of 9 types: Normal, Analysis, Backdoor, DoS, Exploit, Generic, Reconnaissance, Shellcode, and Worm (Moustafa & Slay, 2015b).

AIM

This project aims to create a classification model which can differentiate malicious network traffic from several attack types from packets seen in normal network traffic.

Methodology

CHOOSING A CLASSIFIER

SciKit-Learn not only provides an extremely useful Python library for creating machine learning models – but also has a plentiful supply of documentation to help people decide which algorithm is best suited for their problem. The *"SciKit Learn Algorithm Cheatsheet"* provides a decision tree to help people choose an appropriate algorithm for their intended purpose, as can be seen in Figure 1 (SciKit-Learn, n.d).

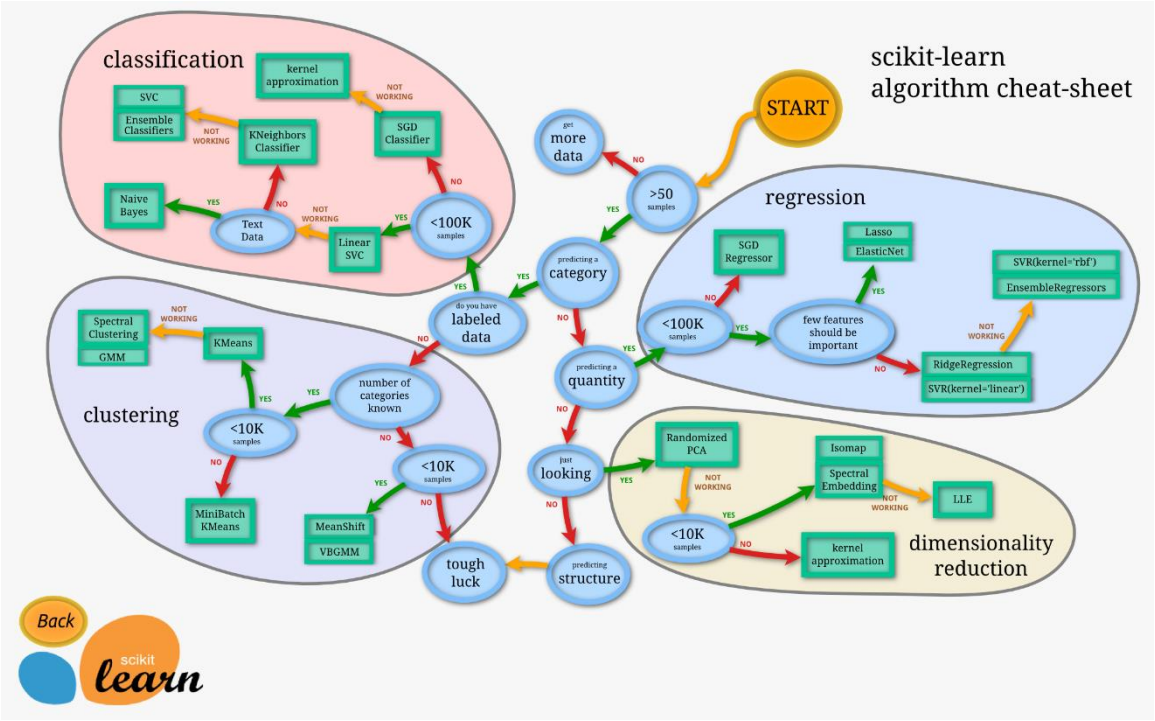

Figure 1 – The SciKit-Learn "Algorithm Cheat-Sheet"

This project aims to classify network packets by attack type, as such this problem will require a classification algorithm. The UNSW-NB15 dataset provided for this project contains 82,332 packet samples in its training set which are labelled by attack type and can be encoded in a numerical format using encoders. By analysing the *"score"* of each model, which is how well it predicts labels when given its training data, the most appropriate algorithm can be found. Ideally, the number outputted by score should be as close to 1, or 100%, as possible. This project will utilise the Random Forest Classifier algorithm from SciKit-Learn's ensemble classifiers.

PROCESSING DATA

To use the UNSW NB15 data set with the Random Forest Classifier, non-numerical data needed to be converted into a numerical format. Firstly, both training and testing data sets were concatenated to create a common encoding for both datasets. The attack type

was then encoded using the *factorize()* function from the *Pandas* Python library, which assigns a number to each unique string in the field much like SciKit-Learns *LabelEncoder()*, before being split from the rest of the concatenated data set and then split back into training and testing labels.

The rest of the concatenated data was then encoded using the *get_dummies()* function, also from *Pandas*. Like Sci-Kit Learn's *OneHotEncoder(),* this function converts each unique string in a column into its own column which can either be 1 or 0, representing true and false, respectively. The *get_dummies()* function improves on the *OneHotEncoder* by automatically identifying non-numerical fields in the data, while not encoding numerical data.

The issue with the *factorize()* function is some machine learning algorithms will detect the numbers assigned to each item as a pattern (Srinidhi, 2018). The *get_dummies()* function overcomes this issue by assigning a new column to each unique item, which prevents the model from potentially learning each packet characteristic may be part of a pattern related to the characteristics of the previous packet.

ALGORITHM FINE-TUNING

To formulate the optimal settings to use for the algorithm the *GridSearchCV* function was used, which is available as part of SciKit-Learn's *"model_selection"* package (SciKit-Learn, n.d). The cross-validator tested 36 models to identify the most accurate yet lightweight solution, changing the *"n_estimators"* and *"max_depth"* parameters of the random forest classifier, as seen below in Figure 2.

```
parameters = {
        "n_estimators":[5,10,50,100,250,1000],
        "max_depth":[2,4,8,16,32,None]
    }
```

Figure 2 - Settings for Parameter Testing

Another valid way of fine-tuning the training of the model is to remove unnecessary data from the dataset. Using the *feature_importances* function from a model trained on the whole data set will show each feature used by the model to categorise data, along with a percentage of how important it rates it.

ASSESSING MODEL PERFORMANCE

To assess the performance of the trained model, a variety of performance metrics can be generated. This includes confusion matrices, Precision-Recall curves, Receiver Operating Characteristic curves, and calculating accuracy of the model predictions.

A confusion matrix illustrates the test results in the form of a matrix. In a binary confusion matrix, predictions can either be true or false. To plot a binary confusion matrix for this

classifier, *"Normal"* labelled packets will represent a negative value while all other values will represent a positive value. These are then plotted as true of false, dependent on if the model classified them correctly.

The Receiver Operating Characteristic (ROC) curve is used to illustrate the True-Positive Rate (TPR) against the False-Positive Rate (FPR) using the binary data processed above. Ideally, this should rise as quickly as possible to 1 which maximises the area under the curve (AUC). If plotted with data from a perfect classifier, would rise straight to 1 due to the lack of false-positives and therefore have an AUC value of 1. By definition, the larger area under the curve, the better the model is at the classification task.

Precision quantifies the number of correct positive predictions, while recall quantifies the proportion of actual positives that were identified correctly. Using this curve, the classification threshold of the model can be adjusted to decrease the number of false-negatives produced if necessary, at the cost of increasing the number of false-positives.

Results

Printing the feature importance's of the data from the Random Forest Classifier Showed no obvious unimportant features to trim from test data. Importance's ranged from 7% to 0%, which is not significant enough to justify scrapping features from the data set. The output from the *feature_importances* function can be seen in Appendix A - Feature Importances.

The random forest classification algorithm was trained using *"n_estimators=250"*, *"max_depth=32"* as per the findings of the cross-validator, as well as a randomisation seed of 49. The full output from the cross validator can be seen in Appendix B - Parameter Testing Output.

Using the *"score"* function – which assesses the accuracy of the model's predictions against its training data – it was found that the model achieved an average score of 93.58% across 5 runs. The model successfully categorised 75.8% of the test data set, illustrated below in Figure 3 as a matrix of each packets' predicted versus actual category. The decoded attack categories can be seen in Appendix C -Decoded Attack Categories. A more traditional binary confusion matrix, classifying non-attack packets as negative and attack packets as positive, can be seen in Table 1 – Binary Confusion Matrix for Random Forest Classifier.

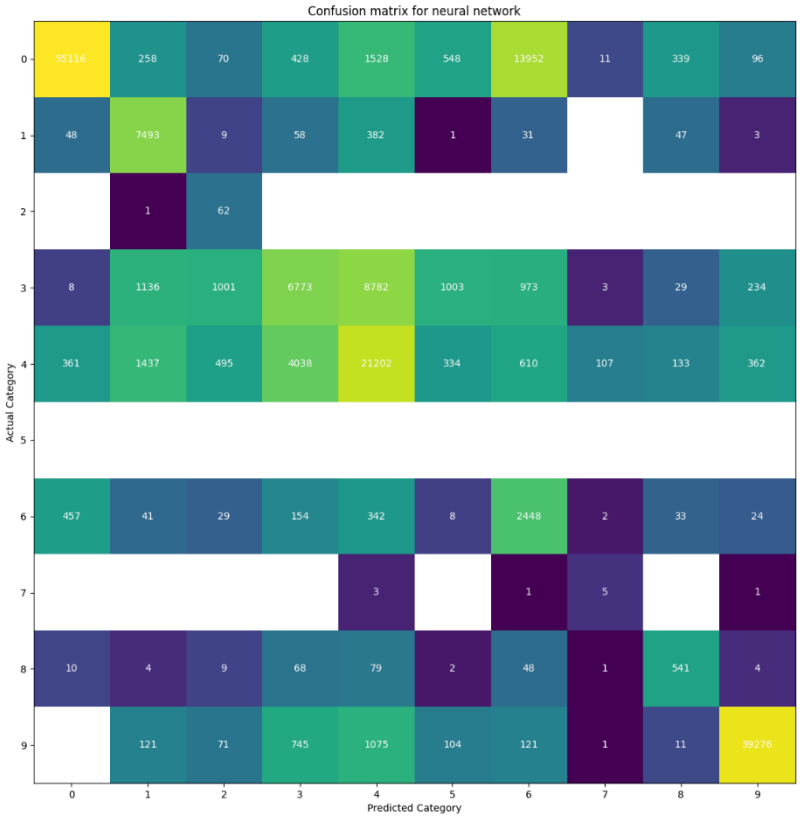

Figure 3 - Confusion Matrix of Each Packets' Predicted Versus Actual Category

	Predicted Negative	Predicted Positive
Negative (Normal-Classified)	55116	884
Positive (Attack-Classified)	17230	102111

Table 1 – Binary Confusion Matrix for Random Forest Classifier

Table 1 presents the test result data in a far easier to understand format and outlines that, out of the 119,341 packets not classified as *"Normal"*, 17,230 packets managed to bypass the model. Using this data, the binary accuracy of the model was calculated as 89.67% using the following equation (Google, Inc., 2020).

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

The receiver operating characteristic (ROC) and precision-recall curves generated from test data for the neural network model can be seen below in Figure 4 and Figure 5 respectively.

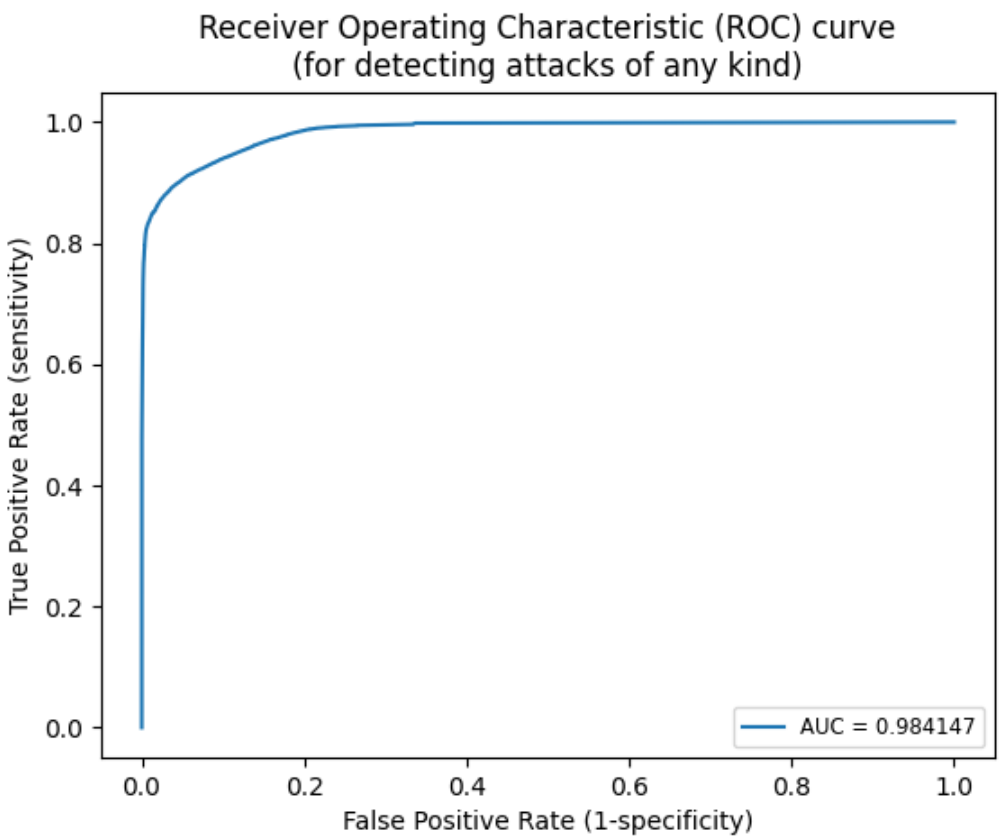

Figure 4 - Receiver Operating Characteristic Curve

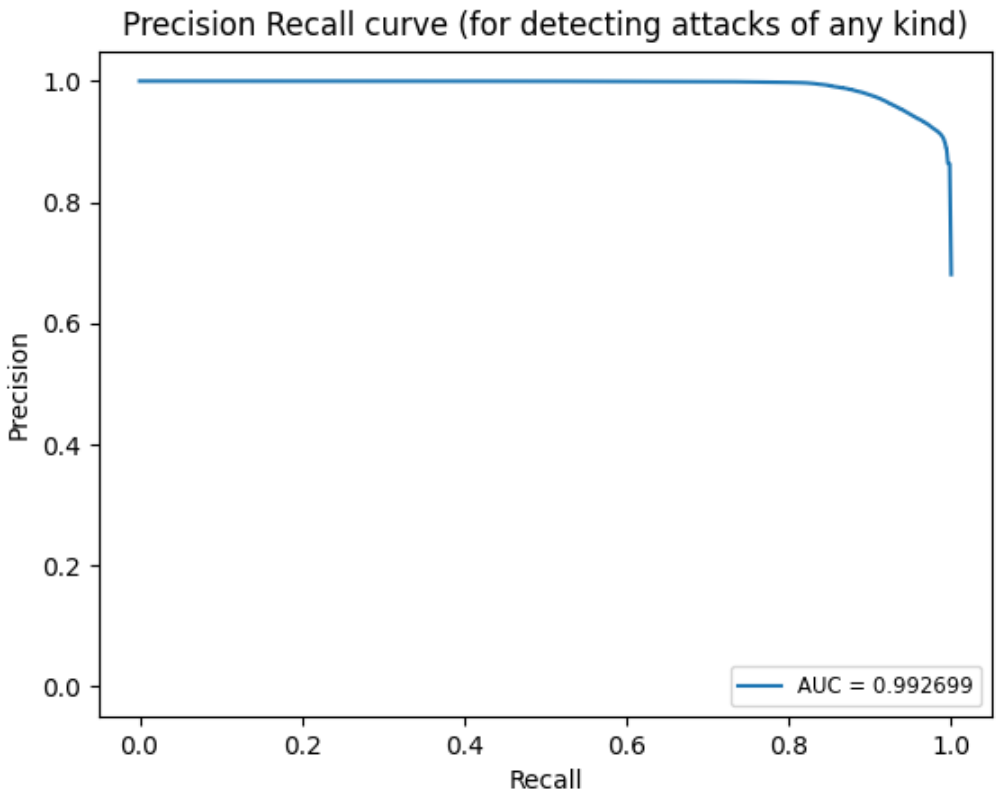

Figure 5 - Precision Recall Curve

Conclusion

Overall, the trained random forest classifier performed well during testing. Referring to the aim discussed in the introductory chapter, the model can successfully differentiate malicious network traffic from normal traffic with a 90% accuracy differentiating normal from attack packets.

As can be seen in Figure 3, the classifier had trouble differentiating Fuzzer packets from Normal packets, which contributed to most false-negatives. A Fuzzer attack is classified as *"an attack in which the attacker attempts to discover security loopholes in an application, operating system or a network by feeding it with the massive inputting of random data to make it crash"* (Moustafa & Slay, 2015b).

Improvements could be made by adjusting the threshold of the model to lower the number of false-negative classifications by the model at the expense of increasing the number false-positives. This would be acceptable given the current low value of false positives from testing data. The default threshold for a random forest classifier is 0.5 which, when predicting the probable attack labels of test packets as opposed to the actual labels, resulted in similar results to actual predictions as can be seen in Table 2. When this threshold is reduced to 0.25, the false-negative rate of the model decreased substantially at the cost of a much higher false-positive rate, as seen in Table 3. This improved the overall binary accuracy of the classifier when predicting the probability of labels from 90.4% to 92.4%, decreasing the number of false negatives by 35%.

	Predicted Negative	**Predicted Positive**
Negative	54717	1283
Positive	15431	103910

Table 2 - Predicted Probable Labels for Test Packets at 0.5 Threshold

	Predicted Negative	**Predicted Positive**
Negative	52468	3532
Positive	10036	109305

Table 3 - Predicted Probable Labels for Test Packets at 0.25 Threshold

Further parameter tweaking could be carried out by adjusting more parameters using the *GridSearchCV* function, which will help to further improve the predictive accuracy of the model.

Works Cited

Google, Inc., 2020. *Classification: Accuracy.* [Online]
Available at: https://developers.google.com/machine-learning/crash-course/classification/accuracy
[Accessed 30 April 2020].

Moustafa, N. & Slay, J., 2015b. *The significant features of the UNSW-NB15 and the KDD99 Data sets for Network Intrusion Detection Systems,* Kyoto: IEEE.

Moustafa, N. & Slay, J., 2015. UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In: *2015 Military Communications and Information Systems Conference (MilCIS).* Canberra: IEEE, pp. 1-6.

SciKit-Learn, n.d. *3.2.4.3.1. sklearn.ensemble.RandomForestClassifier.* [Online]
Available at: https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html
[Accessed 30 April 2020].

SciKit-Learn, n.d. *Choosing the right estimator.* [Online]
Available at: https://scikit-learn.org/stable/tutorial/machine_learning_map/index.html
[Accessed 29 April 2020].

SciKit-Learn, n.d. *sklearn.model_selection.GridSearchCV.* [Online]
Available at: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html
[Accessed 29 April 2020].

Srinidhi, S., 2018. *Label Encoder vs. One Hot Encoder in Machine Learning.* [Online]
Available at: https://medium.com/@contactsunny/label-encoder-vs-one-hot-encoder-in-machine-learning-3fc273365621
[Accessed 29 April 2020].

Srivastava, T., 2014. *How does Artificial Neural Network (ANN) algorithm work? Simplified!.* [Online]
Available at: https://www.analyticsvidhya.com/blog/2014/10/ann-work-simplified/
[Accessed 30 April 2020].

Appendices

Appendix A - Feature Importances

```
Variable: state                 Importance: 0.08
Variable: tcprtt                Importance: 0.05
Variable: ct_state_ttl          Importance: 0.05
Variable: ct_dst_ltm            Importance: 0.05
Variable: ct_ftp_cmd            Importance: 0.05
Variable: rate                  Importance: 0.04
Variable: dur                   Importance: 0.03
Variable: spkts                 Importance: 0.03
Variable: dpkts                 Importance: 0.03
Variable: sbytes                Importance: 0.03
Variable: sttl                  Importance: 0.03
Variable: synack                Importance: 0.03
Variable: dmean                 Importance: 0.03
Variable: ct_srv_src            Importance: 0.03
Variable: service               Importance: 0.02
Variable: dbytes                Importance: 0.02
Variable: dload                 Importance: 0.02
Variable: sloss                 Importance: 0.02
Variable: dloss                 Importance: 0.02
Variable: stcpb                 Importance: 0.02
Variable: dtcpb                 Importance: 0.02
Variable: trans_depth           Importance: 0.02
Variable: proto                 Importance: 0.01
Variable: dttl                  Importance: 0.01
Variable: sload                 Importance: 0.01
Variable: sinpkt                Importance: 0.01
Variable: dinpkt                Importance: 0.01
Variable: sjit                  Importance: 0.01
Variable: djit                  Importance: 0.01
Variable: dwin                  Importance: 0.01
Variable: response_body_len     Importance: 0.01
Variable: is_ftp_login          Importance: 0.01
Variable: swin                  Importance: 0.0
Variable: ackdat                Importance: 0.0
Variable: smean                 Importance: 0.0
Variable: ct_src_dport_ltm      Importance: 0.0
Variable: ct_dst_sport_ltm      Importance: 0.0
Variable: ct_dst_src_ltm        Importance: 0.0
Variable: ct_flw_http_mthd      Importance: 0.0
Variable: ct_src_ltm            Importance: 0.0
Variable: ct_srv_dst            Importance: 0.0
Variable: is_sm_ips_ports       Importance: 0.0
```

Appendix B - Parameter Testing Output

```
Testing Parameters...
Best parameters are: {'max_depth': 32, 'n_estimators': 250}


0.652 + or -0.025 for the {'max_depth': 2, 'n_estimators': 5}
0.634 + or -0.035 for the {'max_depth': 2, 'n_estimators': 10}
0.632 + or -0.037 for the {'max_depth': 2, 'n_estimators': 50}
0.636 + or -0.035 for the {'max_depth': 2, 'n_estimators': 100}
0.642 + or -0.028 for the {'max_depth': 2, 'n_estimators': 250}
0.641 + or -0.028 for the {'max_depth': 2, 'n_estimators': 1000}
0.735 + or -0.016 for the {'max_depth': 4, 'n_estimators': 5}
0.739 + or -0.011 for the {'max_depth': 4, 'n_estimators': 10}
0.73 + or -0.01 for the {'max_depth': 4, 'n_estimators': 50}
0.73 + or -0.012 for the {'max_depth': 4, 'n_estimators': 100}
0.736 + or -0.01 for the {'max_depth': 4, 'n_estimators': 250}
0.738 + or -0.012 for the {'max_depth': 4, 'n_estimators': 1000}
0.788 + or -0.017 for the {'max_depth': 8, 'n_estimators': 5}
0.795 + or -0.014 for the {'max_depth': 8, 'n_estimators': 10}
0.793 + or -0.017 for the {'max_depth': 8, 'n_estimators': 50}
0.793 + or -0.017 for the {'max_depth': 8, 'n_estimators': 100}
0.793 + or -0.016 for the {'max_depth': 8, 'n_estimators': 250}
0.792 + or -0.016 for the {'max_depth': 8, 'n_estimators': 1000}
0.818 + or -0.029 for the {'max_depth': 16, 'n_estimators': 5}
0.821 + or -0.026 for the {'max_depth': 16, 'n_estimators': 10}
0.827 + or -0.024 for the {'max_depth': 16, 'n_estimators': 50}
0.828 + or -0.024 for the {'max_depth': 16, 'n_estimators': 100}
0.829 + or -0.024 for the {'max_depth': 16, 'n_estimators': 250}
0.829 + or -0.024 for the {'max_depth': 16, 'n_estimators': 1000}
0.819 + or -0.028 for the {'max_depth': 32, 'n_estimators': 5}
0.829 + or -0.027 for the {'max_depth': 32, 'n_estimators': 10}
0.835 + or -0.028 for the {'max_depth': 32, 'n_estimators': 50}
0.835 + or -0.028 for the {'max_depth': 32, 'n_estimators': 100}
0.836 + or -0.028 for the {'max_depth': 32, 'n_estimators': 250}
0.836 + or -0.028 for the {'max_depth': 32, 'n_estimators': 1000}
0.82 + or -0.028 for the {'max_depth': None, 'n_estimators': 5}
0.829 + or -0.027 for the {'max_depth': None, 'n_estimators': 10}
0.835 + or -0.027 for the {'max_depth': None, 'n_estimators': 50}
0.836 + or -0.028 for the {'max_depth': None, 'n_estimators': 100}
0.836 + or -0.027 for the {'max_depth': None, 'n_estimators': 250}
0.836 + or -0.028 for the {'max_depth': None, 'n_estimators': 1000}
```

Appendix C - Decoded Attack Categories

LABEL	CATEGORY
0	Normal
1	Reconnaissance
2	Backdoor
3	DoS
4	Exploits
5	Analysis
6	Fuzzers
7	Worms
8	Shellcode
9	Generic